C# / .NET / GIT

GIT LEERBOEK & NASLAGWERK

De Git-basis en GitHub-praktijk



VERSIE 1.1 | DEFINITIEF

21 oktober 2026

AUTEURS

Martin Terlouw en ChatGPT

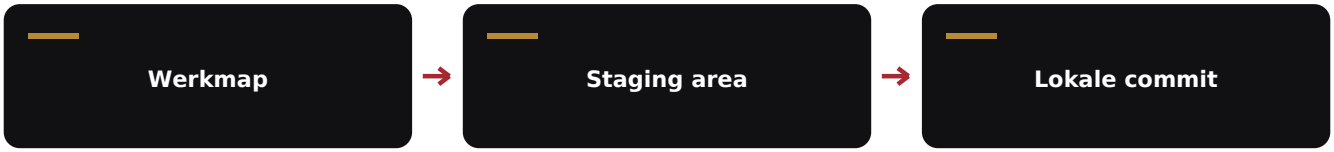
Gebaseerd op Git-uitleg versie 1.1

DOCUMENTOVERZICHT

Een helder naslagwerk voor de Git-basis

Deze presentatieversie is samengesteld uit het goedgekeurde bronbestand `GIT-UITLEG.md`. Het bronbestand blijft leidend voor inhoudelijke wijzigingen; deze PDF is bedoeld voor lezen, presenteren en afdrukken.

DE KERN VAN GIT IN ÉÉN OOGOPSLAG



Documentgegevens

Documentstatus: Definitief — goedgekeurd en gecommit (volgens opgave)

Auteurs: Martin Terlouw en ChatGPT

Hoofdstukken

01 Doel van dit document

02 Wat is Git?

03 Verschil tussen Git en GitHub

04 Wat is een Git-repository?

05 De eerste Git-commando’s

06 Praktijkvoorbeeld eerste commit

07 Waarschuwingen en fouten

08 Hoe wordt dit document bijgehouden?

09 Git en GitHub in huidige werkwijze

10 .gitignore en C#-projectbestanden

11 Mijlpaal Git/GitHub-praktijktoets

12 Vervolg en documentversies

HOOFDSTUK 01

Doel van dit document

Dit document is een persoonlijk leerboek en naslagwerk over Git binnen het C-sharp- en .NET-leertraject. Het legt de behandelde onderwerpen uit in eenvoudige taal, maar bevat genoeg technische informatie om ook zelfstandig door andere ontwikkelaars te worden gelezen.

Het document groeit mee met het leertraject. Versie 1.1 omvat ook GitHub Desktop, online synchronisatie, .gitignore en de praktijktoets van 21 oktober 2026.

HOOFDSTUK 02

Wat is Git?

> Git is een programma dat ervoor zorgt dat je een herstelpunt hebt als er iets in het project fout is gelopen.

Deze vergelijking is een goed vertrekpunt. Git is een programma voor **versiebeheer**. Versiebeheer houdt bij hoe bestanden in een project veranderen. Daardoor ontstaat een geschiedenis waarin terug te vinden is welke wijzigingen zijn vastgelegd en met welke bedoeling dat is gedaan.

Zo'n vastgelegd moment heet een **commit**. Een commit is een momentopname van de wijzigingen die op dat moment bewust zijn geselecteerd. De commit lijkt op een herstelpunt: later kan worden bekeken hoe het project er bij die commit uitzag. Git maakt echter niet automatisch na iedere wijziging een commit. De ontwikkelaar bepaalt zelf welke wijzigingen bij elkaar horen en wanneer ze worden vastgelegd.

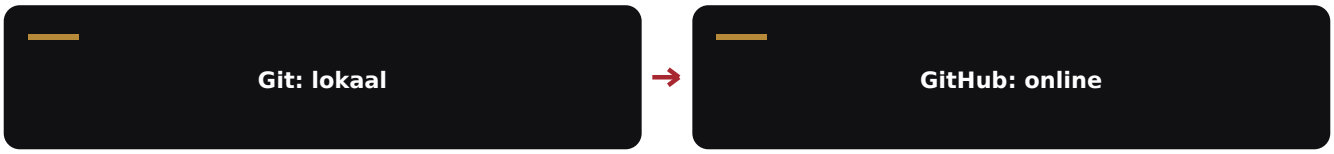
Stel dat een document eerst alleen een titel bevat. Later worden een inleiding en drie hoofdstukken toegevoegd. Als na iedere afgeronde fase een commit wordt gemaakt, kan later worden nagegaan wanneer de inleiding is toegevoegd of hoe het document er vóór het derde hoofdstuk uitzag. Zonder zo'n vastgelegde geschiedenis zou alleen de nieuwste versie direct zichtbaar zijn.

Een goede commit bevat daarom een logisch geheel van wijzigingen en een kort bericht dat uitlegt wat er is vastgelegd.

HOOFDSTUK 03

Verschil tussen Git en GitHub

TWEE VERSCHILLENDE ROLLEN



> Git is als een lokale bibliotheek op mijn computer, GitHub is als een bibliotheek op het internet.

Git werkt lokaal op de computer. De repository, commits en geschiedenis kunnen zonder internetverbinding worden gebruikt. Een ontwikkelaar kan lokaal wijzigingen bekijken, bestanden klaarzetten en commits maken.

GitHub is een online dienst waarop Git-repositories kunnen worden opgeslagen. Een online repository kan worden gebruikt als extra opslagplaats en om code te delen of samen te werken met andere ontwikkelaars.

Git is het lokale versiebeheersysteem; GitHub bewaart repositories online.
Een lokale commit staat niet automatisch op GitHub. De geschiedenis wordt bewust gepubliceerd via push. Dit is inmiddels uitgevoerd met GitHub Desktop.

HOOFDSTUK 04

Wat is een Git-repository?

Een **repository** is een projectmap waarvan Git de geschiedenis bijhoudt. De repository bevat de gewone projectbestanden en een verborgen map met Git-gegevens. In die verborgen gegevens bewaart Git onder andere commits, instellingen en verwijzingen naar de geschiedenis.

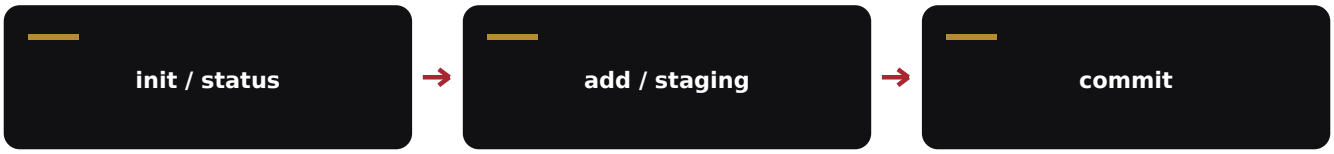
Binnen dit leertraject is `E:\Leren Programeren` de praktijkmap. Nadat Git daar is geïnitialiseerd, is deze map de lokale repository. De documentatie staat in de submap `docs` of `Docs`; op Windows verwijzen die schrijfwijzen naar dezelfde map, terwijl Git de vastgelegde schrijfwijze in zijn geschiedenis bewaart.

De standaardbranch heet `main`. Een **branch** is een ontwikkellijn binnen de geschiedenis. Voor les 0 is alleen van belang dat `main` de naam is van de huidige hoofdontwikkellijn. Werken met meerdere branches komt later aan bod.

HOOFDSTUK 05

De eerste Git-commando's

DE BASISCOMMANDO'S IN VOLGORDE



5.1 `git init`

Met `git init` wordt van de huidige map een Git-repository gemaakt.

```
cd "E:\Leren Programeren"
git init
```

Git maakt hiermee de interne gegevensstructuur voor versiebeheer. Het commando maakt nog geen commit en legt nog geen projectbestand vast.

5.2 `git status`

Met `git status` wordt de huidige toestand van de repository opgevraagd.

```
git status
```

Dit commando kan onder andere laten zien:

- op welke branch wordt gewerkt;
- welke bestanden nog niet door Git worden gevolgd;
- welke bestanden zijn gewijzigd;
- welke wijzigingen klaarstaan voor de volgende commit.

Een **untracked file** is een bestand dat in de projectmap staat, maar nog niet door Git wordt gevolgd. Git neemt zo'n bestand pas mee wanneer het bewust aan de staging area wordt toegevoegd.

5.3 `git add docs/`

Met `git add` worden wijzigingen geselecteerd voor de volgende commit. Dit heet **staging**.

```
git add docs/
```

In dit project selecteert het commando de bestanden en wijzigingen in de documentatiemap. Staging is een tussenstap: de wijzigingen staan klaar, maar zijn nog niet als commit in de geschiedenis vastgelegd.

`git add` en `git commit` hebben dus verschillende taken:

- `git add` kiest welke wijzigingen bij de volgende momentopname horen;
- `git commit` legt de gekozen wijzigingen definitief vast in de lokale Git-geschiedenis.

Na `git add` kan opnieuw `git status` worden gebruikt om te controleren wat klaarstaat.

5.4 `git commit`

Met `git commit` worden de staged wijzigingen vastgelegd. De optie `-m` voegt direct een commitbericht toe.

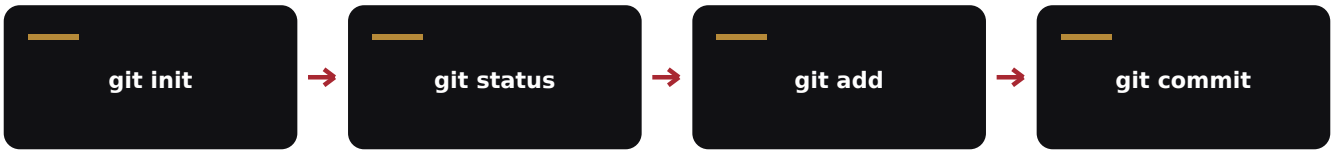
```
git commit -m "Startdocumentatie versie 1.0"
```

Een **commitbericht** is een korte omschrijving van de vastgelegde wijziging. Het bericht moet later begrijpelijk maken wat de bedoeling van de commit was. `Startdocumentatie versie 1.0` geeft bijvoorbeeld aan dat de eerste versie van de startdocumentatie is vastgelegd.

HOOFDSTUK 06

Praktijkvoorbeeld van de eerste commit

VOORBEELD VAN EEN EERSTE COMMIT



De eerste vastlegging van dit leertraject is stap voor stap uitgevoerd. Eerst is de werklocatie als repository geïnitialiseerd:

```
cd "E:\Leren Programeren"
git init
```

Daarna is de toestand van de repository gecontroleerd:

```
git status
```

Vervolgens is de documentatiemap klaargezet voor de eerste commit:

```
git add docs/
```

Daarna is de startdocumentatie vastgelegd. De aanwezige Git-geschiedenis toont als daadwerkelijk commitbericht `Startdocumentatie 1.0`:

```
git commit -m "Startdocumentatie 1.0"
```

De repository bevestigt de volgende feiten:

- de branch heet `main`;
- de eerste commit heeft het bericht `Startdocumentatie 1.0`;
- `Docs/StartDocumentatie.md` is in die commit toegevoegd;
- de commit heeft geen oudercommit en is daarom de eerste commit van de geschiedenis.

Git kan de eerste commit aanduiden als een **root commit**. Root betekent hier het beginpunt van de commitgeschiedenis. Deze commit heeft geen eerdere commit waarnaar hij terugverwijst.

Er wordt hier geen volledige terminaluitvoer gereconstrueerd. Alleen commando's en gegevens die bij de uitgevoerde stappen of in de huidige repository zijn vastgesteld, zijn opgenomen.

HOOFDSTUK 07

Waarschuwingen en fouten

Een **foutmelding** betekent dat Git een opdracht niet of niet volledig kon uitvoeren. De oorzaak moet dan worden onderzocht voordat ervan wordt uitgegaan dat de opdracht is geslaagd.

Een **waarschuwing** wijst op iets dat aandacht verdient, maar betekent niet automatisch dat de opdracht is mislukt. Git kan bijvoorbeeld de volgende waarschuwing tonen:

```
LF will be replaced by CRLF
```

Een tekstbestand bestaat uit regels. Tussen die regels staan onzichtbare tekens die het einde van een regel aangeven. Die tekens heten **regeleinden**.

- `LF` is een regeleinde dat veel wordt gebruikt op Linux en macOS.
- `CRLF` is het gebruikelijke regeleinde op Windows.

De waarschuwing betekent dat Git verwacht een bestand met `LF` bij gebruik op Windows om te zetten naar `CRLF`. Het bestand en de commit zijn daarmee niet automatisch mislukt. Git meldt alleen dat de schrijfwijze van de regeleinden kan veranderen. Bij twijfel wordt na het commando `git status` gebruikt om te controleren welke toestand Git werkelijk ziet.

Het onderscheid is belangrijk: een waarschuwing wordt gelezen en beoordeeld; een foutmelding wordt onderzocht en opgelost voordat verder wordt gegaan.

HOOFDSTUK 08

Hoe wordt dit document bijgehouden?

Dit document groeit mee als persoonlijk leerboek en technisch naslagwerk. Alleen onderwerpen die daadwerkelijk in het leertraject zijn behandeld, worden toegevoegd. Daardoor blijft zichtbaar welke kennis al aan bod is gekomen en welke uitleg als basis geldt voor volgende lessen.

Nieuwe informatie wordt in het passende genummerde hoofdstuk geplaatst of krijgt een nieuw hoofdstuk wanneer dat duidelijker is. Voorbeelden worden gekoppeld aan de eigen repository en worden niet aangevuld met verzonnen uitvoer. Bij een inhoudelijke wijziging wordt het versienummer aangepast wanneer dat binnen het leertraject is afgesproken.

HOOFDSTUK 09

Git en GitHub in de huidige werkwijze

WERKWIJZE IN ÉÉN OOGOPSLAG



Git beheert lokale wijzigingen en commits. GitHub bewaart de gepushte repository online en maakt werken op meerdere computers mogelijk. GitHub Desktop is de grafische toepassing die onder meer commitgeschiedenis, fetch, pull en push toegankelijk maakt; het is geen vervanging van Git of GitHub.

De leerrepository is zeosei/Leren-Programeren, op branch main. In dit project zijn onder andere Docs/, Coding/ en .gitignore opgenomen. De herziene Docs/StartDocumentatie.md versie 1.3 is op 21 oktober 2026 lokaal gecommit en vervolgens via GitHub Desktop gepusht. De gebruiker heeft op GitHub de versie 1.3 en het bijbehorende commitbericht gecontroleerd.

Voor de laptop is GitHub Desktop al aanwezig. Het binnenhalen en controleren van de leerrepository op de laptop is nog niet bevestigd. GitHub synchroniseert de bestanden en commits, maar niet automatisch Visual Studio, de .NET SDK of alle persoonlijke IDE-instellingen.

9.1 Stappen die al zijn uitgevoerd

1. git status gebruiken om wijzigingen te bekijken.
2. git add Docs/StartDocumentatie.md gebruiken om één bestand klaar te zetten.
3. git add . gebruiken om alle niet-genegeerde wijzigingen binnen de huidige map en submappen te stagen; dit is krachtig, maar vereist extra controle.
4. Nogmaals git status gebruiken om de staging area te inspecteren.
5. git commit -m "beschrijvend bericht" uitvoeren. Bij berichten met spaties zijn aanhalingstekens nodig.
6. In GitHub Desktop de juiste repository kiezen en Push origin gebruiken.
7. Op GitHub controleren of de bedoelde wijziging online staat.

Een schone werkboom (nothing to commit, working tree clean) zegt iets over de lokale toestand en bewijst op zichzelf niet dat wijzigingen naar GitHub zijn gepusht.

9.2 Samenwerken tussen vaste pc en laptop

HOOFDSTUK 09

Git en GitHub in de huidige werkwijze (vervolg)

De eerste keer wordt de bestaande repository via File → Clone repository in GitHub Desktop op de laptop binnengehaald. Maak daarvoor geen nieuwe repository aan. Bij een volgende werksessie: eerst Fetch origin, indien nodig Pull origin, dan pas wijzigingen maken. Na het werk: wijzigingen controleren, committen en Push origin. Wissel pas van computer wanneer de bedoelde commits online staan. Bij conflicten niet blind overschrijven, maar eerst onderzoeken.

HOOFDSTUK 10

.gitignore en C#-projectbestanden

WERKWIJZE IN ÉÉN OOGOPSLAG



Bij het toevoegen van het eerste C#-project stond Coding/ aanvankelijk onder Untracked files. Er is een .gitignore gemaakt om gegenereerde Visual Studio- en .NET-bestanden buiten versiebeheer te houden. Bij de controle met git status --short --untracked-files=all waren de solution (.slnx), projectdefinitie (.csproj) en broncode (Program.cs) zichtbaar, zonder bin, obj of .vs in de lijst.

Een gebruikelijke basis is:

```
**/bin/
**/obj/
.vs/
```

Een .gitignore voorkomt niet automatisch dat bestanden die al gevolgd werden uit Git verdwijnen; daarvoor is een afzonderlijke bewuste actie nodig.

HOOFDSTUK 11

Mijlpaal — zelfstandige Git/GitHub-praktijktoets

WERKWIJZE IN ÉÉN OOGOPSLAG



Datum: 21 oktober 2026

Onderdeel: softwareontwikkeling, Git en GitHub; niet de ondernemersles.

Eindbeoordeling: 9/10, door de gebruiker geaccepteerd.

De praktijktoets betrof de aparte repository zeosei/Bedrijfs-voering. Bij de controle werden een publieke repository, branch main, twee commits, een README en een map PDF Documents met de bedoelde PDF aangetroffen.

De gebruiker heeft zelfstandig het volgende werkproces beschreven en uitgevoerd: git status → git add . → opnieuw git status → git commit → publiceren met GitHub Desktop. Hiermee zijn het onderscheid tussen lokale staging en commit, en het online publiceren via GitHub, praktisch toegepast.

De eerste beoordeling was 8,5/10. De definitieve, geaccepteerde mijlpaalbeoordeling werd 9/10. Verbeterpunten voor een volgende oefening zijn een informatievere README en specifiekere commitberichten. Het cijfer betekent dat deze praktijkmijlpaal is behaald; het is geen bewijs dat alle geavanceerde Git-onderwerpen beheerst worden.

HOOFDSTUK 12

Vervolg en documentversies

WERKWIJZE IN ÉÉN OOGOPSLAG



De volgende Git-oefening richt zich op het gecontroleerd ophalen van dezelfde repository op de laptop, de verschillen tussen Fetch en Pull, en het correct wisselen tussen beide computers. Daarna kunnen onderwerpen als branches, mergeconflicten en herstelacties aan bod komen wanneer de basis voldoende is getoetst.

Versiehistorie

- 1.0 — 16 oktober 2026: goedgekeurde Git-basis en eerste commit.
- 1.1 — 21 oktober 2026: voorstel tot actualisering met GitHub Desktop, online synchronisatie, .gitignore, praktijkmijlpaal 9/10 en vervolgafspraken.